

Uma framework integrada para o desenvolvimento de CRDTs

João Garção

Universidade NOVA de Lisboa & NOVA Lincs

Resumo Os *Conflict-Free Replicated Data Types* (CRDTs) representam uma abstração fundamental para garantir *Strong Eventual Consistency* (SEC) em sistemas distribuídos modernos, permitindo a atualização concorrente de dados em múltiplas réplicas sem coordenação centralizada. Contudo, a construção correta destas estruturas é um desafio de engenharia propenso a erros que podem comprometer a integridade da aplicação. Embora as atuais ferramentas de verificação automática baseadas em *solvers* SMT, como o VeriF_x, permitam a criação rápida de protótipos e a validação de propriedades algébricas de convergência, estas enfrentam limitações significativas de expressividade ao lidar com definições recursivas e invariantes de estado globais complexas. Por outro lado, plataformas de verificação dedutiva, como o Why3, oferecem o rigor necessário para promover uma semântica *correct-by-construction*, porém, exigem ao programador um maior domínio da linguagem.

Procurando unir o melhor das duas abordagens, este artigo propõe a implementação de uma nova *framework* para o desenvolvimento formal e modular de CRDTs. A abordagem apresentada introduz uma nova linguagem para a definição destas estruturas, que, após rigorosa validação semântica e tipificação na sua Árvore de Sintaxe Abstrata (AST), traduz automaticamente o código para diferentes ambientes de verificação. Em particular, detalhamos o mecanismo de tradução *source-to-source* que gera abstrações funcionais para o VeriF_x e teorias formais em linguagem WhyML. Esta arquitetura permite conciliar a automação da verificação de convergência à robustez das provas dedutivas de invariantes, ocultando a complexidade das ferramentas matemáticas e reduzindo o esforço exigido ao programador.

Keywords: CRDTs · Verificação Formal · Why3 · VeriF_x

1 Introdução

Os sistemas distribuídos modernos estão sujeitos a requisitos rigorosos de disponibilidade e baixa latência que, na presença de partições de rede, impossibilitam a garantia de consistência forte. O compromisso estabelecido pelo Teorema CAP obriga a adoção de modelos de consistência mais relaxados, como a *Strong Eventual Consistency* (SEC). Neste contexto, os *Conflict-Free Replicated Data Types* (CRDTs) [5] assumem um papel central, permitindo que diferentes réplicas sejam

atualizadas de forma concorrente e sem necessidade de coordenação centralizada, garantindo matematicamente a convergência do estado.

Apesar destas propriedades teóricas, o *design* e implementação corretos de CRDTs em aplicações reais continuam a ser um desafio. Basta uma falha na lógica da função de combinação de estado para resultar na divergência permanente ou ir contra a intenção do utilizador. Para mitigar estes problemas, o recurso à verificação formal tem-se revelado uma necessidade, dividindo-se o estado da arte em duas abordagens principais: a verificação automática e a verificação dedutiva.

Ferramentas automatizadas com recurso a SMT *solvers*, como a linguagem VeriF_x [2], são altamente eficazes na validação rápida de propriedades algébricas de convergência. No entanto, esbarram em limites de expressividade quando necessitam de raciocinar sobre estruturas recursivas não limitadas ou sobre invariantes de estado globais. Por outro lado, plataformas de verificação dedutiva como o Why3 [1] oferecem o poder expressivo necessário para promover semânticas *correct-by-construction*, recorrendo a contratos explícitos e *ghost code*. A grande desvantagem desta abordagem reside no esforço manual exigido e na exigente aprendizagem imposta aos programadores.

A motivação do trabalho aqui apresentado surge da necessidade de preencher esta lacuna metodológica. Propomos uma *framework* integrada para o desenvolvimento de CRDTs, com o objetivo de aliar a conveniência da verificação automática à robustez das provas dedutivas. Para concretizar esta integração, desenhou-se uma nova linguagem que captura a semântica dos CRDTs. O desenho de uma linguagem própria, em vez da conversão direta de VeriF_x para Why3, permite ocultar as complexidades das linguagens alvo.

Neste artigo, focamo-nos na apresentação e implementação do *pipeline* de compilação da nossa *framework*. O sistema processa o código fonte através de análise lexical e sintática, garante a sua correção através de um sistema de tipos e, por fim, traduz a Árvore de Sintaxe Abstrata (AST) para as ferramentas alvo. Esta estratégia de tradução tem como principal contribuição a ocultar a complexidade inerente à verificação dedutiva, reduzindo drasticamente o esforço manual de especificação exigido ao programador, ao mesmo tempo que mantém garantias matemáticas sobre a convergência e a integridade.

2 Verificação Formal de CRDTs

Para garantir SEC na ocorrência de partições de rede, os CRDTs [3] apoiam-se em propriedades matemáticas que asseguram a convergência determinística das réplicas, independentemente da ordem ou do atraso na entrega de mensagens. O modelo de sincronização baseado em estado (CvRDTs), onde a mensagem enviada contém o estado completo da réplica, exige que o espaço de estados forme um *join-semilattice*, no qual a função de *merge* computa o limite superior mínimo de dois estados, sendo que para garantir a convergência determinística, esta função tem obrigatoriamente de ser comutativa, associativa e idempotente. Em alternativa, o modelo baseado em operações (CmRDTs), foca-se na disseminação individual de operações recebidas, sendo necessário que todas estas possam ser

executadas de forma concorrente, comutem entre si e garantindo que a ordem de aplicação não afeta o estado final [5].

Apesar das propriedades teóricas destas estruturas serem claras, a sua implementação prática é complexa e suscetível a erros que podem comprometer a integridade dos dados, tornando o recurso à verificação formal uma necessidade. Atualmente, a verificação divide-se em duas grandes abordagens com compromissos distintos: a verificação automática baseada em SMTs e a verificação dedutiva.

A verificação automática recorre a linguagens de alto nível, como o VeriF_x [2], que permite a prototipagem de estruturas replicadas através de abstrações funcionais. Esta ferramenta gera fórmulas lógicas que são resolvidas automaticamente pelo *solver* Z3, provando propriedades algébricas, como a SEC. Embora seja rápida, esta abordagem enfrenta limitações críticas de expressividade ao lidar com *designs* complexos.

Especificamente, as ferramentas baseadas puramente em *solvers* SMT têm dificuldade em raciocinar sobre definições recursivas e propriedades que exijam iteração sobre estruturas de tamanho ilimitado. No caso do CRDT *Replicated Growable Array* (RGA), por exemplo, a natureza recursiva da sua lógica de inserção excede frequentemente a capacidade de raciocínio do *solver*, resultando na impossibilidade de concluir a verificação devido a *timeouts*. Além disso, a ferramenta não consegue capturar a intenção semântica das políticas de resolução de conflitos, ou seja, pode provar matematicamente que uma estrutura converge, mesmo que essa convergência produza um comportamento inesperado para o domínio da aplicação, como, por exemplo, aplicar uma semântica *remove-wins* quando se pretendia *add-wins* [4].

Para abordar estas limitações, a verificação dedutiva apresenta-se como uma solução robusta. Plataformas como o Why3 [1], que se baseia na linguagem de programação e especificação de primeira ordem WhyML, oferecem um nível de expressividade e estabilidade muito superior no raciocínio de estruturas de dados aninhadas ou recursivas. O Why3 permite impor uma disciplina estática rigorosa através da definição de pré- e pós-condições, invariantes de ciclo, e garantias de terminação. Em algoritmos complexos, o Why3 suporta ainda a introdução de *ghost code*, que consiste em dados e computações auxiliares que servem exclusivamente para fundamentar a prova matemática, sem impactar o fluxo de controlo do programa executável final. Através destas anotações, a plataforma gera obrigações de prova que são posteriormente verificadas por uma variedade de *provers* externos.

O grande obstáculo da verificação dedutiva é o elevado esforço manual necessário e o conhecimento técnico exigido para guiar as provas de forma interativa. Torna-se, assim, evidente a necessidade de desenhar metodologias que consigam explorar a automatização rápida de ferramentas como o VeriF_x sem abdicar da garantia de correção semântica e manutenção de invariantes complexos que o Why3 possibilita.

3 Breve Descrição da Arquitetura da Framework

A arquitetura da *framework* proposta assenta num modelo de compilação *source-to-source* modular, desenhado para unificar as metodologias de verificação automática e dedutiva num único fluxo de trabalho. O objetivo principal deste modelo é isolar o programador da complexidade sintática e dos detalhes de implementação intrínsecas a ferramentas como o VeriF_x e o Why3. Para tal, a *framework* introduz uma nova linguagem desenhada exclusivamente para a abstração e especificação de CRDTs, bem como dos seus respetivos invariantes.

O processo de tradução inicia-se com a submissão do código fonte, escrito nesta linguagem, ao *frontend* do compilador. Este módulo é responsável pelas fases clássicas de processamento de linguagens: efetua a análise lexical e sintática para garantir a conformidade gramatical e, de seguida, constrói uma representação intermédia sob a forma de uma AST. Antes de qualquer tradução ser efetuada, a AST é submetida a um analisador semântico e a um sistema de tipos rigoroso. Esta etapa de validação estática é importante para a robustez da *framework*, pois garante que as regras da linguagem e as dependências estruturais do CRDT estão corretas, rejeitando especificações inválidas e evitando a geração de código alvo malformado.

Uma vez validada, a representação interna do CRDT é encaminhada para o *backend* da *framework*, que se ramifica em geradores de código independentes, direcionados a diferentes plataformas de verificação. Do ponto de vista arquitetural, o sistema prevê dois alvos de tradução complementares:

Módulo de Geração para VeriF_x Responsável por traduzir a especificação para abstrações funcionais compatíveis com a sintaxe do VeriF_x, visando a prova totalmente automática de propriedades algébricas de convergência, como comutatividade, associatividade e idempotência, através do *solver* Z3.

Módulo de Geração para Why3 Focado em garantir a correção semântica das operações e a preservação de invariantes de estado globais. Este módulo traduz a AST validada para a linguagem WhyML, introduzindo automaticamente pré-condições, pós-condições e axiomas necessários para que a plataforma de verificação dedutiva consiga gerar as correspondentes obrigações de prova.

Ao centralizar a especificação num único ponto e delegar a verificação para as ferramentas mais adequadas a cada classe de propriedades, a *framework* facilita a definição de uma semântica rigorosa. Isto permite que o foco do programador permaneça exclusivamente na lógica e na semântica de resolução de conflitos da aplicação.

4 Pipeline de Compilação para Verificação Formal

O *pipeline* de compilação da *framework* recorre a uma arquitetura clássica de tradução *source-to-source*, dividida em *frontend*, responsável pelo processamento

e validação da linguagem, e em *backend*, que gera o código para as plataformas de destino [2,1]. Esta separação modular garante que a validação lógica e semântica das estruturas de dados replicadas seja feita de forma independente da ferramenta de verificação final.

4.1 Análise Lexical, Sintática e Semântica

A transformação do código escrito na linguagem para uma representação formal constitui o *input* para o *frontend*. O texto introduzido pelo programador é primeiramente processado pelo analisador lexical, que o segmenta numa sequência de *tokens*. Este processo reconhece as palavras-chave da estrutura da nova linguagem, tais como `module`, `interface`, `invariant` e `axiom`, guardando também anotações e atributos de prova, visto que serão indispensáveis para a etapa de verificação.

Para ilustrar o *frontend* mencionado, o seguinte excerto demonstra como o programador define a estrutura e as propriedades de um CRDT na nossa linguagem:

```
module GCounter : Cvrtd
interface
  val increment (v: integer) : payload
end
axiom commutative(merge) [@proof]
```

Esta sequência contínua de *tokens* passa posteriormente pelo analisador sintático para construir a AST. Esta estrutura de dados intermédia modela o programa de forma hierárquica, isolando as assinaturas de definição das interfaces das implementações concretas dos módulos dos CRDTs.

Com a arquitetura da AST definida, a especificação é submetida a uma fase de controlo semântico, garantindo a sua correção antes de transitar para os módulos de tradução. O módulo de tipificação percorre recursivamente os nós da árvore, avaliando as expressões com base nas regras do domínio: assegura a coerência dos tipos em operações matemáticas e de comparação, verifica se as chamadas de operações respeitam o número de argumentos esperado na assinatura do módulo, e garante que as referências a variáveis ou funções fantasmas estão devidamente declaradas e contextualizadas. O resultado final desta análise é uma AST tipificada e logicamente validada, impedindo que o compilador processe código malformado e assegurando que apenas semânticas robustas são encaminhadas para a geração de código.

4.2 Geração de Código

O *backend* desta *framework* recebe a AST validada e encaminha a geração de código para módulos específicos para cada plataforma de destino. O sistema suporta a tradução simultânea para ambas as ferramentas de verificação, aplicando otimizações específicas a cada uma:

Geração para VeriF_x: A tradução para a linguagem VeriF_x atua de forma dinâmica, analisando a assinatura de interface dos módulos processados para inferir o modelo de sincronização pretendido. Se a assinatura identificar um modelo baseado em estados (CvRDT), o tradutor gera classes que estendem o *trait* genérico CvRDT. Por outro lado, num modelo baseado em operações (CmRDT), a geração estende o *trait* CmRDT e os respetivos métodos de propagação de mensagens, tais como `prepare` e `effect` [5]. Uma característica do tradutor é a sua capacidade para inferir estruturas complexas: o sistema identifica automaticamente estados compostos e estruturas vetoriais, típicas em relógios lógicos e identificadores de réplicas, gerando iteradores nativos e funções otimizadas. Axiomas lógicos declarados nesta linguagem são convertidos de forma direta em blocos `proof` do VeriF_x. O axioma de comutatividade definido anteriormente, por exemplo, é traduzido nativamente pelo nosso gerador da seguinte forma, especificando a propriedade a provar e garantindo total automatização na verificação:

```
proof commutative {
  forall (x: T, y: T) {
    (x.reachable() && y.reachable() && x.compatible(y)) =>: {
      x.merge(y).equals(y.merge(x)) &&
      x.merge(y).reachable()
    }
  }
}
```

Como é possível observar neste bloco de código gerado, as primitivas `reachable()` e `compatible()` atuam como restrições sintáticas, que garantem que o *solver* apenas teste a comutatividade da função *merge* sobre estados que podem ser formalmente alcançados a partir do estado inicial, evitando a verificação de estados inválidos.

Geração para Why3: Em paralelo, o gerador de código para a plataforma Why3 converte os tipos abstratos e as especificações lógicas para a sintaxe WhyML. Para respeitar as exigências do sistema de tipos e a necessidade de modularidade da ferramenta, o processo de tradução aplica uma divisão estrutural do código: o tradutor gera módulos auxiliares onde a representação interna do estado e as operações puras são definidas. Estes módulos são então importados e utilizados pelos módulos principais do CRDT. Esta estratégia de separação em dois módulos permite abstrair funções genéricas como o *merge*, expondo axiomas e propriedades como invariantes de estado. O módulo principal do compilador coordena a execução deste *pipeline*, garantindo que, numa execução bem-sucedida, são materializados os múltiplos ficheiros finais, prontos a ser avaliados pelos respetivos *solvers*.

5 Casos de Estudo

Esta secção foca-se em dois casos de estudo principais: por um lado, demonstrar a capacidade de expressividade da linguagem de *frontend* no suporte a diferentes

modelos de sincronização, baseados em estado e em operações, e por outro, quantificar a redução do esforço de especificação manual exigida ao programador.

Para ilustrar o processo de compilação, consideramos primeiramente a implementação de um contador estritamente crescente baseado em estado (*Grow-Only Counter*). Neste modelo, o programador interage exclusivamente com a nossa linguagem, onde define a estrutura do CRDT [5], a sua interface e a semântica de resolução de conflitos:

```
module GCounter_CRDT : Cvrtdt
  type payload = integer
  val init_state: payload = 0
  val merge (a b: payload) : payload = max(a, b)
  val compare (a b: payload) : boolean = a == b
  val equals (a b: payload) : boolean = compare(a, b) && compare(b, a)
  val increment (v: integer, a: payload) : payload =
    requires { v > 0 }
    a + v
end
```

Neste excerto, a condição `requires { v > 0 }` garante que o contador apenas aceita valores positivos. Além disso, a função `compare` representa a relação de ordem ($\leq$) do *join-semilattice*. Por esse motivo, a definição de `equals` reflete a propriedade de ordem parcial da estrutura ($a \leq b \wedge b \leq a \implies a = b$).

A partir desta especificação, o *backend* atua de forma dinâmica para derivar os modelos lógicos para os dois ambientes de destino. Para a ferramenta VeriF_x [2], o sistema infere o modelo de sincronização, gerando a classe que contém a lógica do CRDT e que estende `CvRDT`, integrando as propriedades algébricas exigidas pelo *solver* através do objeto de prova `CvRDTProof`:

```
class GCounter_CRDT(payload: Int) extends CvRDT[GCounter_CRDT] {
  def increment(v: Int): GCounter_CRDT = {
    new GCounter_CRDT(this.payload + v)
  }
  def value(): Int = { this.payload }
  private def max(a: Int, b: Int): Int = { if (a >= b) a else b }
  def merge(that: GCounter_CRDT): GCounter_CRDT = {
    new GCounter_CRDT(this.max(this.payload, that.payload))
  }
  def compare(that: GCounter_CRDT): Boolean = { this.payload == that.payload }
}
object GCounter_CRDT extends CvRDTProof[GCounter_CRDT]
```

Ao mesmo tempo, o tradutor de código para a plataforma Why3 [1] preserva a mesma semântica, mas respeitando as restrições rigorosas do sistema de tipos e modularidade impostas pela verificação dedutiva. A ferramenta encarrega-se de separar a lógica automaticamente, isolando a representação de estado e as

funções base num módulo auxiliar, o qual é posteriormente importado pelo módulo principal que expõe a teoria formal do CRDT para cumprir a assinatura exigida:

```

module GCAuxiliary
  use int.Int, int.MinMax
  type payload = int
  type t = { payload: int; }
  let function get_payload (a: t) : payload = a.payload
  let ghost function create () : t = { payload = 0 }
  let function merge (a b: t) : t = { payload = max a.payload b.payload }
  predicate compare (a b: t) = a.payload = b.payload
  let function increment (v: int) (a: t) : t = { payload = a.payload + v }
end

module GCounter : Cvrtdt
  use int.Int, int.MinMax
  use GCAuxiliary as GCAux
  type payload = GCAux.payload
  type t = GCAux.t
  let function get_payload (a: t) : int = GCAux.get_payload a
  let ghost function create () : t = GCAux.create ()
  let function merge (a b: t) : t = GCAux.merge a b
  predicate compare (a b: t) = GCAux.compare a b
  predicate equals (a b: t) = compare a b /\ compare b a
end

```

Para além do *Grow-only Counter*, a eficácia da *framework* foi avaliada na tradução de estruturas baseadas em operações, como por exemplo um *Counter* baseado em operações, em estruturas de estado compostas, como é o caso do *Positive-Negative Counter*, e em estruturas baseadas em conjuntos, como o *Grow-Only Set* (GSet) e o *Two-Phase Set* (TwoPSet). O caso do *Positive-Negative Counter* ilustra a capacidade de composição do sistema para suportar tanto incrementos quanto decrementos, reaproveitando diretamente a lógica e a estrutura do *GCounter_CRDT* previamente definido, e mantendo o estado através de duas instâncias independentes deste contador base para registar as adições e as subtrações.

A Tabela 1 sintetiza o esforço de especificação exigido ao programador em comparação com a quantidade de código gerado pela ferramenta. A métrica de Linhas de Código (LoC) exclui as definições das interfaces globais da *framework*, quantificando apenas a dimensão do código gerado específico de cada CRDT.

Como se pode observar na Tabela 1, a linguagem desenhada permite expressar o comportamento da estrutura tendencialmente em menos linhas. Mais importante do que a redução do número de linhas é a capacidade do compilador para gerar código sintaticamente distinto, adaptado às particularidades de cada ferramenta. No caso do *PNCCounter*, o programador define a lógica de composição em 25 linhas, delegando ao compilador o reaproveitamento dos módulos

CRDT	Modelo	LoC Escritas	LoC (VeriF _x)	LoC (WhyML)
G_Counter	Estado	13	24	33
PN_Counter	Estado	25	26	26
Op_Counter	Operações	14	17	19
GSet	Estado	16	9	37
TwoPSet	Estado	24	16	44
TwoPSet	Operações	32	24	28

Tabela 1. Sumário do esforço de especificação e código gerado por CRDT.

auxiliares já existentes do *GCounter* no Why3 e a derivação das dependências de tipificação no VeriF_x.

Os ficheiros finais produzidos nestes casos de estudo foram submetidos com sucesso às respetivas plataformas. Os *solvers* automáticos, como o Alt-Ergo no Why3 e o Z3 no VeriF_x e no Why3, foram capazes de processar os ficheiros e validar as obrigações de prova geradas. Esta validação prática demonstra que a arquitetura *source-to-source* desenvolvida liberta o programador das exigências de verificação formal.

Adicionalmente, embora os casos de estudo ilustrados se centrem em contadores, toda a estrutura de compilação para o VeriF_x e para o Why3 foi idealizada para acomodar estruturas recursivas, como o RGA, permitindo combinar a verificação automática ao raciocínio indutivo.

6 Trabalho Relacionado

A verificação formal de CRDTs tem evoluído de provas manuais exaustivas para abordagens suportadas por ferramentas, dividindo-se fundamentalmente em duas vertentes com diferentes compromissos: a verificação automática e a verificação dedutiva.

No âmbito da verificação automática, destacam-se as ferramentas que recorrem a SMT *solvers* para simplificar a prova de propriedades algébricas. A ferramenta VeriF_x [2] representa um passo significativo para tornar esta verificação acessível, oferecendo uma linguagem de alto nível que permite a implementação de CRDTs e a validação automática de *Strong Eventual Consistency* (SEC) através do *solver* Z3. Contudo, as abordagens puramente baseadas em SMT enfrentam limitações severas ao lidar com algoritmos e estruturas de dados recursivas. O *Replicated Growable Array* (RGA), por exemplo, excede frequentemente a capacidade do *solver* para raciocinar sobre caminhos de execução não limitados devido à natureza recursiva da sua lógica de inserção. Adicionalmente, a verificação automática pode apresentar instabilidade face a alterações sintáticas triviais e não garante, por si só, que a resolução de conflitos se alinha com a intenção semântica do programador ou com as invariantes globais da aplicação.

Para ultrapassar as limitações de expressividade das ferramentas puramente automáticas, é frequente o recurso a plataformas de verificação dedutiva como o Why3 [1], que atua como um ambiente de verificação intermédio, gerando

obrigações de prova a partir da linguagem WhyML para múltiplos *solvers* externos. O Why3 impõe uma disciplina estática rigorosa sobre o estado mutável, dispensando modelos de memória complexos e oferecendo uma estabilidade e expressividade que o tornam adequado para lidar com definições recursivas e aninhadas.

Neste sentido, a arquitetura apresentada neste artigo destaca-se por unificar as vantagens de ambos os mundos. Ao invés de forçar o programador a optar entre a automação limitada e a expressividade manual exigente, a nossa abordagem *source-to-source* procura tirar o máximo partido do melhor que estas duas ferramentas têm para oferecer. Assim, combinamos a facilidade e automatização da prova de convergência proporcionada pelo VeriF_x com o rigor do Why3 na imposição de uma semântica *correct-by-construction* em estruturas de dados complexas e recursivas.

Por fim, para que o leitor possa aceder à *framework*, todo o código fonte desenvolvido encontra-se disponível num repositório no GitHub. O projeto pode ser consultado e testado através do nosso repositório, acessível em <https://github.com/JoaoGarcao/master-thesis/tree/main/crdt-verification/framework>.

7 Conclusão e Trabalho Futuro

Neste artigo, apresentámos uma *framework* integrada para o desenvolvimento e verificação formal de *Conflict-Free Replicated Data Types* (CRDTs). A solução proposta aborda o desafio de conciliar a rapidez e automação da verificação automática com a robustez e expressividade das provas dedutivas. Através de uma arquitetura *source-to-source*, a nossa ferramenta permite ao programador definir a estrutura, a interface e a semântica dos CRDTs numa linguagem de alto nível. O compilador garante a validação sintática e semântica da especificação e deriva, de forma automática, os modelos formais para a ferramenta VeriF_x e para a plataforma Why3.

A avaliação realizada demonstrou que a *framework* reduz o esforço manual de especificação exigido ao programador na maioria dos casos. A partir de uma base de código minimalista, o sistema é capaz de gerar artefactos sintaticamente distintos e adaptados às particularidades de cada ferramenta alvo.

Como trabalho futuro, planeamos expandir a expressividade da linguagem de *frontend* e do respetivo sistema de tipos para suportar a composição modular de CRDTs aninhados (*nested CRDTs*). Para além disso, seria interessante explorar a extensão do sistema para centralizar os resultados da verificação das duas ferramentas numa única interface, bem como a integração de *bounded model checkers* para a deteção rápida de contraexemplos. Por fim, pretendemos avaliar a *framework* em algoritmos e estruturas de dados de maior complexidade que dependam de definições recursivas ou que exijam forte raciocínio indutivo.

Agradecimentos. Este trabalho é parcialmente financiado pelo NOVA LINS (UID/04516/2025) com o apoio financeiro da FCT/IP.

Referências

1. Filliâtre, J., Paskevich, A.: Why3 - where programs meet provers. In: Felleisen, M., Gardner, P. (eds.) Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings. Lecture Notes in Computer Science, vol. 7792, pp. 125–128. Springer (2013). https://doi.org/10.1007/978-3-642-37036-6_8, https://doi.org/10.1007/978-3-642-37036-6_8
2. Porre, K.D., Ferreira, C., Boix, E.G.: Verifx: Correct replicated data types for the masses. In: Ali, K., Salvaneschi, G. (eds.) 37th European Conference on Object-Oriented Programming, ECOOP 2023, Seattle, Washington, United States, July 17-21, 2023. LIPIcs, vol. 263, pp. 9:1–9:45. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2023). <https://doi.org/10.4230/LIPICS.ECOOP.2023.9>, <https://doi.org/10.4230/LIPICS.ECOOP.2023.9>
3. Preguiça, N.M.: Conflict-free replicated data types: An overview. CoRR **abs/1806.10254** (2018), <http://arxiv.org/abs/1806.10254>
4. dos Santos Borrego, D.: Verifying and Enforcing Application Constraints in Antidote SQL. Master’s thesis, Universidade NOVA de Lisboa (Portugal) (2022)
5. Shapiro, M., Preguiça, N.M., Baquero, C., Zawirski, M.: Convergent and commutative replicated data types. Bull. EATCS **104**, 67–88 (2011), <http://eatcs.org/beatcs/index.php/beatcs/article/view/120>